

Link para o vídeo: https://youtu.be/MxdP62he_dY

Autores:

- Carolina Michel Ferreira
- Felipe Freitas Silva
- Mateus Campos Caçabuena
- Murilo dos Santos Kasperbauer de Moraes

Roteiro:

[Vídeo inicia com imagem de homem apontando para o espectador]

Mateus: Você programador, cansado de fazer testes e não saber o quão eficaz eles são? Convenhamos, é uma boa prática fazer testes no seu sistema; é quase obrigatório de tanta necessidade que tem. E, vamos combinar, $1 + 1$ e fazer um `assertEquals` esperando 2 não é um teste muito eficaz, né?

[Muda para um slide com foto de Richard Lipton e lâmpadas para simbolizar uma boa ideia]

Mateus: Foi esse mesmo pensamento que Richard Lipton teve e introduziu a seguinte ideia - brilhante ideia - de fazer um teste para testes.

[Slide dá espaço a slide introdutório com plano do vídeo; com os tópicos abordados e autores]

Mateus: Aí tu vai me perguntar: “Um teste para testes?”. Exatamente, um teste que avalia a efetividade dos testes automatizados que já existem no sistema. O nome disso é...

[Transição com suspense]

Mateus: ...Teste de Mutação. E é isso que eu (Mateus Caçabuena), Carolina Ferreira, Felipe Freitas e Murilo dos Santos vamos apresentar pra vocês nesse vídeo, então, fique com a gente.

[Transição com emoji pensante]

Carolina: Mas afinal, depois de toda essa propaganda...

[Aparece imagem de Charles Xavier (Professor X; Líder dos X-Men) pensando com muita concentração]

Carolina: ...o que são testes de mutação? O que são mutantes em testes de software?

[Imagem dos X-Men aparece na tela por um breve momento]

Carolina: Imagine os mutantes dos X-Men, cada um com suas próprias habilidades.

[Saem X-Men, entra imagem de código]

Carolina: Agora, aplique isso a um código de um software. Os 'mutantes' são variações do código original, cada um com uma pequena alteração, como mudanças na lógica, remoção de código ou até mesmo alteração de valores e tipos.

[Animação de código-fonte sendo modificado]

Carolina: A ideia é introduzir essas pequenas mutações no código para ver se os testes conseguem detectá-las. É como um jogo de sobrevivência para os mutantes!

[Animação de um teste detectando uma mutação]

Carolina: Se um teste falhar ao detectar uma mutação, significa que o mutante foi 'morto' pelo teste. Mas se o teste não falhar, o mutante 'sobrevive' e precisa ser eliminado.

[Exemplos de mutações aparecem na tela, como a alteração de operadores aritméticos e lógicos]

Carolina: Os exemplos podem ser simples, como mudar um sinal de adição para subtração, ou trocar um operador lógico por outro.

[Gráfico mostrando a métrica "Mutation Score"]

Carolina: No final do processo, um relatório é gerado, mostrando quais mutantes sobreviveram e quais foram 'mortos' pelos testes. Esse relatório também inclui uma métrica chamada 'Mutation Score', [...]

[Mostrar fórmula para calcular "Mutation Score"]

Carolina: [...] que é a proporção de mutantes 'mortos' em relação ao total de mutantes.

[Gráfico de barras mostrando o "Mutation Score"]

Carolina: Quanto mais alto o Mutation Score, mais eficazes são os testes em detectar mutações. Então, da próxima vez que ouvir falar em teste de mutação, lembre-se dos X-Men [...]

[X-Men voltam a aparecer na tela]

Carolina: [...] e dos mutantes no código, e como os testes trabalham arduamente para eliminar qualquer ameaça!

[Transição com imagem de pessoa pensando]

Felipe: Sabendo então o que são esses testes de mutação, nos resta 1 dúvida; qual sua utilidade? É realmente útil introduzir estas alterações no código? Por que alguém “quereria” [...]

[Imagem de alguém quebrando um código/computador]

Felipe: [...] propositalmente "quebrar" seu código?

[A partir daqui, imagens vão se alterando no fundo para gerar mais engajamento]

Felipe: Bem, os testes de mutação são úteis por diversos motivos, mas principalmente para garantir que nosso software é resistente à mudança e confiável.

Felipe: Imaginemos que um programador novo entrou no projeto e, ao mexer em uma função antiga, se depara com uma condicional que julga que não faz sentido; "nunca será atingido mesmo, vou remover". Ou então que ele encontre uma comparação estilo saldo bancário; "Se a quantidade que quero sacar for maior *ou igual* ao que tenho na conta, posso sacar" e queira alterá-la, para que seja apenas *maior*. Ao fazer isso, porém, pode acontecer de haver cenários que o programador desconhece mas que na verdade já estavam previstos e não deveriam ser alterados. Provavelmente, esses cenários foram descobertos graças aos testes de mutação.

Felipe: Sabendo então dessa utilidade, e implementando esta boa prática no seu programa, provavelmente ainda restam dois pensamentos:

“Como eu nunca havia descoberto isso antes? Que ferramenta incrível! Preciso implementá-la agora.”

“Será que é complexo executar este teste e, além disso, possui um custo grande para implementar no meu código?”

Felipe: Vamos deixar isso com o Murilo que ele vai explicar...

[Transição para slides do Murilo]

Murilo: Eu entendo perfeitamente esse questionamento, pois quando nós, programadores, descobrimos algo que parece incrível, começamos a refletir se realmente é aquilo que parece ser sem algo por baixo dos panos, como um custo elevado para implementação ou se é muito complexo.

Murilo: Podemos pensar nas ferramentas de teste de mutação por exemplo, como o PIT Test, que é uma ferramenta com uma linguagem restrita a Java, porém com funcionalidades gratuitas e pagas, onde as funcionalidades gratuitas suprem necessidades básicas como implementações de teste de mutação em testes unitários de startups ou sistemas menores. Também temos ferramentas mais abrangentes como o Stryker Mutation, que vamos demonstrar o funcionamento simples e rápido pelo próprio playground de teste executado em C#, mas que também opera em JavaScript e TypeScript.

[Transição para slide com imagem de teste unitário padrão e algumas mutações]

Murilo: Agora explicando um pouquinho mais como funciona um teste de mutação; nós temos um teste unitário padrão desenvolvido diretamente pelos desenvolvedores para o sistema verificando suas funcionalidades básicas e nós aplicamos a ferramenta de mutação como por exemplo o Stryker.

Murilo: Digamos que nós temos um sistema onde a gente quer desenvolver uma soma de $1 + 1$ e nós queremos fazer com que esse sistema garanta o resultado de $1 + 1$. Esse é o nosso teste unitário padrão. Baseado nisso, nós passamos a desenvolver testes de mutação pela ferramenta, que desenvolve diversas “mutações”, todos de forma semelhante, visando aperfeiçoar e manter o funcionamento do teste unitário padrão como o teste base. Então, se no teste unitário padrão nós temos “ $1 + 1$ ”, o teste mutation 1 (M1) vai ser o teste no qual o código irá executar “ $1 - 1$ ”, já o teste 2, “ $1 * 1$ ”, o teste 3, “ $1 / 1$ ”, e assim por diante, a depender da complexidade do sistema. Tudo isso visando que cada um dos testes seja “morto” à medida que testes vão sendo desenvolvidos - para que tenhamos certeza de que nosso teste unitário é o teste significativo para o sistema.

[Transição para ferramenta Stryker Mutator]

Murilo: Agora vindo aqui para o site da ferramenta Stryker Mutator, nós encontramos outras opções como, por exemplo, o blog interno onde eles mostram todas as atualizações feitas no Stryker nos últimos tempos - a última atualização foi feita no dia 16 de fevereiro de 2024 - nós temos também uma introdução para JavaScript com toda a documentação adequada para a linguagem, assim como exemplos em C# e até mesmo Scala, uma ferramenta de report de testes e de testes de mutação. Dessa forma, nós conseguimos verificar como funciona o teste do Stryker Mutator.

[Transição para o Playground do Stryker]

Murilo: Também temos o Playground interno, que é uma ferramenta simples e rápida para demonstrarmos o que acontece quando nós criamos um teste de mutação baseado em testes unitários padrões. Então, por exemplo, nós temos aqui um script C# com uma classe calculadora com suas funções de adição, subtração e multiplicação. Do outro lado, nós temos a classe básica de teste para essa calculadora, com *test cases* simples. Ao rodarmos estes testes, a ferramenta retorna de forma completa o resultado; mostra que os 4 testes passaram com sucesso.

[Rodando testes com Stryker e mostrando output unitário e de mutação]

Murilo: Isso vai gerar os quatro testes que foram demonstrados aqui. O número de testes passou, todos os testes passaram. Quando nós rodamos o *Striker* com testes de mutação, ele vai nos gerar um *report* um pouco diferente, onde ele vai adicionar um teste (ele adicionou um teste) e matou o teste mutante, ele adicionou o teste padrão e nós podemos ver que cada vez que ele adiciona um teste, ele mata o teste mutante e mantém o teste novo que foi colocado. No programa em C#, podemos ver que na primeira classe é um $A + B$ para fazer a adição na calculadora. O teste mutante fez um $A - B$ para que ele fizesse o teste onde foi “assassinado”. A mesma coisa para subtração, onde na subtração ele trocou de um teste de subtração para adição e no teste de multiplicação para divisão e é dessa forma como o *Striker* funciona. É uma ferramenta muito completa e simples. O fato dela

abranger de forma complexa tudo o que deveria fazer para outras ferramentas, acaba sendo muito poderoso. Então, nós podemos adicionar isso de forma muito simples, rápida e tem uma documentação muito completa. É uma ferramenta que até hoje está sendo adicionada novas features e questões que podem ser utilizadas. Basicamente, o *Striker Mutator* é uma ferramenta muito indicada para este tipo de questão.

[Transição para slide de finalização; agradecimento]

Felipe: Então foi isso, pessoal, espero que tenhamos ajudado vocês a entenderem o que é um teste de mutação, qual sua utilidade, como ele funciona e providenciado um exemplo concreto de sua utilização através de uma ferramenta especializada.

Felipe: Abaixo do vídeo temos diversos links com mais informações interessantes e outras fontes relevantes.

Felipe: Muito obrigado pela atenção e até uma próxima

Fontes Bibliográficas:

Gonçalves, Paulo. Testes de Mutação. Disponível em:

<https://dev.to/paulogoncalvesr/testes-de-mutacao-1c7p>. Acesso em: 02 maio 2024.

Christian, Pablo. Testes de Mutação: Implemente testes eficazes e de alta qualidade. Disponível em:

<https://pablo-christian.medium.com/testes-de-mutação-%EF%B8%8F-implemente-testes-eficazes-e-de-alta-qualidade-4b6c82ffa3a7>. Acesso em: 02 maio 2024.

Engsoftmoderna. Testes de Mutação. Disponível em:

<https://engsoftmoderna.info/artigos/testes-mutacao.html>. Acesso em: 01 maio 2024.

Franco, Weliton Alves. Teste de Mutação: Aplicação e análise de testes de mutação de interface. Disponível em:

<https://www.eldorado.org.br/blog/teste-de-mutacao-aplicacao-e-analise-de-testes-de-mutacao-de-interface/>. Acesso em: 30 abril 2024.

PITest. Disponível em: <https://pitest.org/>. Acesso em: 29 abril 2024.

Baeldung. Java Mutation Testing with PITest. Disponível em:

<https://www.baeldung.com/java-mutation-testing-with-pitest>. Acesso em: 28 abril 2024.

Mutation Testing. Disponível em: <https://mutation-testing.org/>. Acesso em: 27 abril 2024.

Jarrell, Jeremy. JesterDotNet. Disponível em: <https://github.com/jeremyjarrell/JesterDotNet>. Acesso em: 26 abril 2024.

Stryker Mutator. Disponível em: <https://stryker-mutator.io/>. Acesso em: 25 abril 2024.